# Architecture Decision Records

## ShopEasy — E-Commerce Web Application

*A record of significant technical decisions*

## About This Document

This document captures the significant architectural and implementation decisions made during the development of ShopEasy. Each Architecture Decision Record (ADR) follows a standard format: the context that motivated the decision, the decision itself, alternatives that were considered and rejected, and the consequences (both positive and negative) that flow from the choice.

ADRs are written at the point of decision, not retrospectively, and are treated as immutable once accepted. Superseding a decision means writing a new ADR that references and overrides the older one, rather than editing history. This preserves the reasoning trail for future engineers.

### Index

# ADR-001: Feature-sliced architecture over type-sliced

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

React projects commonly organize files either by type (components/, reducers/, hooks/, services/) or by feature/domain (features/cart/, features/products/). The type-sliced approach is familiar and works well for small applications, but as the number of features grows, related files drift far apart in the folder tree, and cross-feature coupling becomes hard to reason about.

ShopEasy contains six clear business domains: products, cart, wishlist, auth, checkout, and orders. Each has its own UI, state, and API surface. Grouping by type would scatter each domain across five or more directories.

## Decision

Organize the codebase by feature. Each feature under src/features/ owns its components, Redux slice, API endpoints, schemas, and helpers. A single index.js at the root of each feature acts as the public API — only symbols exported from index.js may be imported by other features.

- features/cart/ contains CartPage, CartDrawer, CartItem, cartSlice, and index.js — all cart concerns colocated.
- Truly generic UI (Button, Modal, Skeleton) lives in src/components/. Feature-specific UI stays inside the feature.
- Cross-cutting state (global loader, theme, drawer visibility) lives in stores/common/, not inside any single feature.

## Alternatives Considered

### Type-sliced organization

Group by file type: components/, reducers/, hooks/, services/, pages/. This is the pattern in most React tutorials.

**Rejected because:** It scales poorly. Changing "how the cart works" would touch files in five directories. It also encourages accidental coupling because everything sits in shared folders.

### Nested components/ folder inside each feature

features/cart/components/, features/cart/hooks/, features/cart/api/. A more verbose feature-sliced variant.

**Rejected because:** Overkill for this scale. Each feature has 3–6 files; a flat structure inside the feature folder is easier to scan.

## Consequences

### Positive

- Adding a new feature is a self-contained change: create a folder, expose an index.js, register the slice and route.

- Deleting a feature is safe: delete the folder, remove the two references. No orphaned files scattered across the codebase.
- The public API pattern (index.js) prevents accidental deep imports and clarifies which parts of a feature are stable.
- New engineers can understand one feature at a time without reading the whole codebase.

### Negative / Trade-offs

- Slight learning curve for engineers used to type-sliced React projects.
- Requires discipline: developers must remember to export via index.js rather than deep-importing.

# ADR-002: Redux Toolkit with createSlice over classic Redux

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

Classic Redux requires action-type constants, hand-written action creators, and switch-based reducers, typically spread across three files per feature. Redux Toolkit (RTK) consolidates these into a single createSlice call that auto-generates action types and creators.

The Redux team has officially recommended RTK as the standard way to write Redux logic since 2019. Classic Redux remains supported but is considered legacy for new code.

## Decision

Use Redux Toolkit's createSlice for all client-state slices (auth, cart, wishlist, orders, common).

Each feature owns its slice file inside its own directory: features/cart/cartSlice.js, features/auth/authSlice.js, etc.

Cross-cutting actions such as logoutAndClear and resetApp are defined in stores/common/commonActions.js using createAction, and feature slices subscribe via extraReducers.

## Alternatives Considered

### Classic Redux with hand-written reducers

Type constants in constants.js, action creators in commonActions.js, switch-statement reducer in commonReducer.js.

**Rejected because:** Verbose, error-prone (typos in string constants), and no longer the recommended pattern. Reviewers reading modern code expect RTK.

### Zustand or Jotai for client state

Lightweight alternatives to Redux with less boilerplate.

**Rejected because:** The project brief specifies Redux Toolkit. Also, RTK Query (chosen for server state) integrates naturally with Redux, and mixing state libraries adds complexity.

## Consequences

### Positive

- Roughly 60% less boilerplate per slice compared to classic Redux.
- Immer under the hood allows "mutating" syntax in reducers, which reads more naturally.
- Action types are auto-generated from slice name + reducer name, eliminating typo bugs.
- DevTools integration, middleware setup, and serializable-check are configured by default.

### Negative / Trade-offs

- Immer's mutating syntax can confuse engineers who have only used classic Redux; the mental model shift is small but real.

- Deep object mutations inside a slice can still produce subtle bugs if Immer's draft semantics are misunderstood.

# ADR-003: RTK Query over TanStack React Query

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

The application needs a data-fetching layer with caching, request deduplication, background refetching, and loading/error state management. The two mainstream options are RTK Query (bundled with Redux Toolkit) and TanStack React Query.

Both libraries solve the same problem well. React Query is arguably more ergonomic for pure data fetching, while RTK Query is deeply integrated with Redux and reuses the same store infrastructure.

## Decision

Use RTK Query for all server state (product listing, product detail, categories, authentication).

A single baseApi is defined in stores/baseApi.js. Each feature injects its own endpoints via injectEndpoints in its own api file (features/products/productsApi.js, features/auth/authApi.js).

Tag types (Products, Cart, User, Auth) are declared centrally on the base API so any feature can invalidate cross-feature caches.

## Alternatives Considered

### TanStack React Query

A dedicated data-fetching library with excellent devtools and a large ecosystem.

**Rejected because:** Would introduce a second state system alongside Redux. The application already uses Redux for cart/wishlist/auth; adding React Query means two devtools, two mental models, two configuration surfaces. RTK Query does the same job inside the existing store.

### Raw fetch with useEffect

Manually manage loading, error, and caching in each component.

**Rejected because:** Reinvents caching, deduplication, and refetching. Not production-grade.

### axios with a custom hook

A thin useApi hook wrapping axios calls.

**Rejected because:** Same problem as raw fetch — no caching or invalidation. axios also adds a dependency for functionality fetch already provides.

## Consequences

### Positive

- Zero additional dependencies beyond Redux Toolkit.
- Automatic caching, deduplication, and refetching for free.
- Loading and error states available as booleans on every hook.
- Cache invalidation via tags is declarative and centralized.

### Negative / Trade-offs

- RTK Query's API surface is larger than React Query's; there is a learning curve for developers new to it.
- Debugging cache behavior sometimes requires the Redux DevTools inspector, which is less polished than React Query's dedicated devtools panel.

# ADR-004: redux-persist scoped to auth, cart, and wishlist only

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

The application has no backend, so any state a user expects to survive a page reload must be persisted client-side. However, persisting everything indiscriminately (including RTK Query's cache) causes stale data bugs: users would see cached product listings from days ago instead of fresh data.

## Decision

Configure redux-persist with an explicit whitelist: [auth, cart, wishlist].

The RTK Query api slice is explicitly excluded so server data is always fetched fresh on app boot.

The common slice (ephemeral UI state such as drawer open/closed) is also excluded — a page reload should not restore the cart drawer to its open state.

localStorage is used as the storage backend; sessionStorage would lose cart contents when the tab closes.

## Alternatives Considered

### Persist the entire Redux store

Default redux-persist configuration with no whitelist.

**Rejected because:** Would persist RTK Query cache, producing stale product data. Would also persist UI state (drawer visibility) which is not meaningful across sessions.

### Manual persistence via a subscription to store changes

Write custom logic that syncs specific slices to localStorage on every state change.

**Rejected because:** Reinvents redux-persist. Error-prone. redux-persist handles rehydration timing, serialization, and PersistGate cleanly.

### IndexedDB via redux-persist

Use IndexedDB for larger capacity.

**Rejected because:** Cart and wishlist data are small (under 100KB in realistic use). localStorage is sufficient and simpler.

## Consequences

### Positive

- Cart, wishlist, and login persist across page reloads and browser restarts as users expect.
- Product catalog data always reflects the latest DummyJSON response, no stale-cache bugs.
- PersistGate handles the rehydration timing so components never render with undefined persisted state.

### Negative / Trade-offs

- localStorage disabled or unavailable (private browsing on some browsers) breaks persistence silently. Mitigated by wrapping persist logic in defensive code paths.
- Serialization of Redux actions on hydrate/persist triggers Redux's serializable-check warnings unless configured to ignore persist actions.

# ADR-005: Razorpay test-mode integration with client-side success fallback

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

The assignment brief requires a checkout flow ending in payment. Options ranged from a fully mocked in-app form to a real payment gateway integration in test mode. A test-mode integration would show reviewers that the developer can wire up a real payment SDK, which is a meaningful signal.

Razorpay was chosen over Stripe due to its dominance in the Indian market and its test-mode support without full merchant onboarding. However, Razorpay's test mode still requires KYC verification for full end-to-end success responses. Without completed KYC, Razorpay's hosted checkout modal accepts test card details but returns a failure state on submission.

## Decision

Integrate Razorpay Checkout.js in test mode using a publishable test key stored in VITE_RAZORPAY_KEY_ID.

On checkout, invoke the Razorpay modal so the user experiences the real payment UI and can enter test card details.

Because KYC is incomplete, Razorpay returns a failure response. On failure, redirect the user back to the checkout page.

On the checkout page, treat the redirect as a successful transaction: mark the order as placed, clear the cart, and route to the order confirmation page.

Document this behavior clearly in the README and in code comments so reviewers understand the KYC limitation and the intentional client-side success fallback.

## Alternatives Considered

### Fully mocked payment form

Build a custom card form with react-hook-form and zod (Luhn validation), simulate a 2-second processing delay, then succeed.

**Rejected because:** Would not demonstrate real gateway integration experience. Chosen approach shows both the SDK integration and pragmatic handling of a real-world constraint (incomplete KYC).

### Stripe test mode

Use Stripe's Elements or PaymentElement in test mode.

**Rejected because:** Stripe requires a PaymentIntent created server-side for a fully working demo. Without a backend, this requires a similar workaround. Razorpay was more accessible for a frontend-only assignment.

### Complete Razorpay KYC

Finish KYC verification to enable full test-mode success responses.

**Rejected because:** KYC requires personal documentation and takes 1–2 business days. Not feasible within the assignment timeline.

## Consequences

### Positive

- Reviewers see a real payment SDK invocation and hosted checkout modal, not a hand-rolled fake form.
- The user journey (open modal → enter card → return to app → see confirmation) mirrors production payment flows.
- README documents the KYC limitation transparently, demonstrating engineering judgment.

### Negative / Trade-offs

- The failure-to-success client-side override is not something that would ship to production. Requires clear documentation to avoid confusing reviewers.
- A malicious user could theoretically skip the payment step entirely by manipulating client state; not a concern for a demo application with no real transactions, but explicitly called out in the README.
- Depends on Razorpay Checkout.js remaining available and its API stable.

# ADR-006: Shimmer/skeleton loaders over spinner-based loading

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

Every data-fetching view needs a loading state. The two common patterns are: (1) a centered spinner that replaces content, and (2) skeleton/shimmer placeholders that match the shape of the incoming content.

Research from platforms like Facebook, LinkedIn, and YouTube shows skeleton loaders reduce perceived load time compared to spinners because users see the layout stabilize immediately, even before real data arrives.

## Decision

Implement skeleton components for every list, card, and detail view: ProductCardSkeleton, ProductDetailSkeleton, CartItemSkeleton.

Skeletons match the dimensions of the actual content so layout does not shift on data arrival.

A subtle CSS shimmer animation (background-position transition on a linear gradient) conveys activity.

Reserve spinners only for inline button-level loading states, such as "Placing order..." during checkout submission.

## Alternatives Considered

### Full-page spinners

A centered spinner replaces the entire content area during load.

**Rejected because:** Causes layout shift on data arrival. Feels slower even when actual load times are identical. Modern e-commerce sites do not use full-page spinners for content loads.

### No loading state (progressive rendering)

Render whatever data is available and update as more arrives.

**Rejected because:** RTK Query does not stream partial responses; data arrives in one payload. Empty screens during load look broken.

### A prebuilt skeleton library (react-loading-skeleton)

Use a dependency for skeleton primitives.

**Rejected because:** Skeletons are simple CSS. Adding a dependency for what is essentially a div with a gradient animation is unnecessary.

## Consequences

### Positive

- Perceived performance improves; users see the page structure immediately.
- No layout shift when data arrives, improving Cumulative Layout Shift metrics.
- Skeletons are simple, dependency-free CSS.

### Negative / Trade-offs

- Requires maintaining a skeleton variant for each major content component. Divergence between skeleton and real component can lead to visible jumps.

# ADR-007: Cart drawer alongside dedicated cart page

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

Users interact with the cart in two distinct contexts: (1) quickly verifying an add-to-cart action or making a small tweak while browsing, and (2) reviewing the full cart before checkout. A single UI cannot serve both use cases well.

Amazon, Flipkart, and most major e-commerce platforms provide both a "mini cart" (drawer or popover) and a dedicated cart page.

## Decision

Implement both a slide-in cart drawer (CartDrawer) and a full cart page (CartPage) that share the same underlying cart state.

The drawer opens from the right when the user clicks the cart icon in the navigation. It shows all cart items, subtotal, and a "Go to Cart" button plus a "Checkout" button.

The drawer's open/closed state is stored in commonSlice (isCartDrawerOpen) so any component can toggle it.

The cart page (/cart) shows the same items with more space for editing quantities and viewing the full price breakdown.

Both surfaces read from the same cartSlice; there is no duplication of state.

## Alternatives Considered

### Drawer only, no dedicated page

All cart interactions happen in the slide-in drawer.

**Rejected because:** Drawers are cramped for cart review. Users cannot easily share a cart URL, and long carts scroll awkwardly inside a narrow drawer.

### Cart page only, no drawer

Clicking the cart icon navigates to /cart.

**Rejected because:** A full-page navigation for a quick "did that add?" check is heavy. Users lose their browsing context.

### Popover from the cart icon

A dropdown-style popover instead of a full-height drawer.

**Rejected because:** On mobile, popovers are cramped. A drawer transitions naturally to a full-height sheet on smaller screens.

## Consequences

### Positive

- Users get lightweight interactions (drawer) and full review (page) as appropriate to their intent.
- Cart drawer keeps browsing context — the user can add, verify, and continue shopping without leaving the current page.
- Single source of truth (cartSlice) ensures the two surfaces never disagree.

### Negative / Trade-offs

- Two UI surfaces to maintain. Design changes to cart items must be applied consistently.
- On very small screens, the drawer behaves more like a bottom sheet, requiring specific responsive handling.

# ADR-008: Debounced product search

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

The product search input triggers an API call to DummyJSON. Without debouncing, every keystroke would issue a request. A user typing "headphones" would generate ten calls, most of which are wasted because the intermediate results are irrelevant.

DummyJSON has no rate limits published, but the pattern is wasteful regardless — it congests the network, spikes the browser's request queue, and can produce out-of-order responses where an earlier request resolves after a later one.

## Decision

Implement a useDebounce custom hook in src/hooks/useDebounce.js that returns a value delayed by a configurable interval.

Wrap the search input's value with useDebounce(searchTerm, 300).

RTK Query's getProducts query subscribes to the debounced value, not the raw input. This means keystrokes update the input immediately (visually responsive) but only fire an API call after the user stops typing for 300ms.

The 300ms threshold is a common industry standard: short enough that users don't feel lag, long enough to eliminate most intermediate requests.

## Alternatives Considered

### No debouncing

Every keystroke triggers a request.

**Rejected because:** Wasteful. Produces network congestion and potential out-of-order response bugs.

### Throttling instead of debouncing

Limit calls to one every N milliseconds regardless of typing pattern.

**Rejected because:** Throttling still fires calls during typing. Debouncing waits for the user to stop, which is the correct semantic for search.

### A search button that triggers the query on click

User types then clicks "Search".

**Rejected because:** Modern users expect live search. A button adds friction and feels dated.

### A lodash.debounce wrapper

Import debounce from lodash.

**Rejected because:** Adds a dependency for logic that fits in a 10-line custom hook. The hook version integrates cleanly with React's lifecycle.

## Consequences

### Positive

- API call volume reduced by roughly 80–90% during typing.
- No out-of-order response bugs.
- Input remains visually responsive to keystrokes; only the API call is delayed.

### Negative / Trade-offs

- A 300ms delay between last keystroke and result display. Users typing very quickly may perceive a brief pause. This is standard behavior on all major e-commerce sites.

# ADR-009: Toast notifications for all user-triggered state changes

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

Users take actions (add to cart, remove from wishlist, log in, place order) and need immediate confirmation that the action succeeded. Without feedback, users often repeat actions ("did that add? let me click again"), producing duplicate items or duplicate submissions.

The two common feedback patterns are (1) inline confirmation (a "Added!" label next to the button that fades out) and (2) toast notifications (a message that appears in a corner of the viewport briefly).

## Decision

Use react-toastify for all user-triggered success and error feedback.

A single ToastContainer is mounted once in MainLayout so toasts appear consistently across all pages.

Toast semantics: success (green) for confirmations, error (red) for failures, info (blue) for neutral notices such as "Item already in cart".

Toasts auto-dismiss after 3 seconds by default.

Actions that emit toasts: add/remove from cart, add/remove from wishlist, login success/failure, logout, order placement.

## Alternatives Considered

### Inline confirmations

A small badge or label near the triggered action.

**Rejected because:** Users often look away from the button after clicking (to the cart icon, for example). Inline feedback is easy to miss.

### Alert() or window.confirm

Native browser dialogs.

**Rejected because:** Modal, blocking, ugly, and cannot be styled. Universally rejected as poor UX in modern apps.

### Hand-rolled toast component

Build a custom Toast primitive in components/.

**Rejected because:** react-toastify is small, battle-tested, and handles queueing, positioning, accessibility, and animations. Reinventing this is not a good use of assignment time.

## Consequences

### Positive

- Every user action produces immediate, visible feedback.
- Reduces duplicate submissions and user confusion.

- react-toastify handles positioning, animation, and accessibility (screen-reader announcements).

#### Negative / Trade-offs

- Overuse can annoy users. Discipline is required to only toast on meaningful actions, not every state update.
- One additional dependency, though small (about 12KB gzipped).

# ADR-010: Explicit empty, loading, and error states for every data view

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

Every view that fetches or displays data has four possible states: (1) loading, (2) loaded with data, (3) loaded with no data, (4) error. Handling only the "happy path" (state 2) is a common source of broken user experiences.

Assignments and prototypes frequently skip empty and error states, but production applications treat them as first-class concerns. Reviewers scan for this handling as a signal of production-thinking.

## Decision

Every data-fetching view must render appropriately in all four states.

- Loading: skeleton components matching the target content layout (see ADR-006).
- Empty: a shared EmptyState component in src/components/EmptyState/ displaying an icon, headline, subtext, and (where applicable) a primary action such as "Continue Shopping" or "Clear Filters".
- Error: a headline, friendly message, and a "Try Again" button that re-triggers the query.
- Data: the actual content.

Views with empty states: product listing (no matches for filters/search), cart (no items), wishlist (no items), order history (no past orders).

## Alternatives Considered

### Show a blank screen when empty

If there is no data, render nothing.

**Rejected because:** Users perceive blank screens as broken. They will reload the page or leave.

### A single generic "No data" message

One string reused everywhere.

**Rejected because:** Different empty contexts need different messaging. "No products match your filters" and "Your cart is empty" call for different actions ("Clear filters" vs "Continue shopping").

## Consequences

### Positive

- Users always see a meaningful screen, never a broken-looking blank one.
- Error states guide users toward recovery (retry button) instead of leaving them stuck.
- Signals production-grade thinking to reviewers.

### Negative / Trade-offs

- More components to build and maintain (EmptyState with variants).

- Discipline required — it is tempting to skip empty states during rapid development.

# ADR-011: Environment-specific config files (dev / nonprod / prod)

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

Even in a frontend-only assignment, real applications distinguish between development, staging (nonprod), and production environments. Different environments may need different API URLs, feature flags, log verbosity, and third-party keys.

Vite supports this natively via .env.<mode> files and the --mode flag on build/dev commands.

## Decision

Maintain three environment files at the project root: .env.development, .env.nonprod, .env.production.

A shared .env file holds defaults, and a git-ignored .env.local supports per-developer overrides.

A committed .env.example documents required variables without values.

All environment access flows through a single module src/config/env.js that reads import.meta.env, validates required variables at startup (throwing if any are missing), and exposes helpers such as isDev, isNonProd, isProd.

package.json exposes scripts for each mode: dev, start:nonprod, build:dev, build:nonprod, build:prod.

## Alternatives Considered

### A single .env file with runtime environment detection

One file with all variables, and code that switches behavior based on hostname or a global.

**Rejected because:** Hardcoding all values in a single file conflates environments. Runtime detection is fragile.

### Hardcoded configuration inside src/

A constants.js file with values that change per environment before each build.

**Rejected because:** Requires editing source code for every environment switch. High risk of accidentally shipping dev values to production.

## Consequences

### Positive

- Clear separation between environments; impossible to accidentally use dev values in a prod build.
- Startup validation catches missing variables early instead of producing confusing runtime errors.
- Centralized env access means changing an env variable's name requires editing only one file.

### Negative / Trade-offs

- Slight setup overhead compared to a single .env file.
- Developers must remember to update all three env files when adding a new variable, plus .env.example.

# ADR-012: Plain CSS with design tokens over SCSS or CSS-in-JS

| Status | Accepted | Date | 05 July 2026 |
|---|---|---|---|

## Context

Styling options for a modern React app include: SCSS, plain CSS with custom properties, CSS Modules, Tailwind, and CSS-in-JS libraries (styled-components, Emotion). Each has trade-offs in tooling, bundle size, runtime cost, and developer experience.

The application needs a design token system (colors, spacing, typography) and per-feature styling. Runtime CSS-in-JS was ruled out early due to its bundle-size and performance cost. The remaining decision was between SCSS and plain CSS.

## Decision

Use plain CSS with CSS custom properties (variables) for design tokens.

Design tokens live in styles/tokens.css and are declared on :root, making them accessible everywhere and trivially themeable (dark mode = one property override on :root).

Per-feature stylesheets (products.css, cart.css, checkout.css) organize styles by domain.

A single styles/main.css imports all other stylesheets in the correct cascade order, and main.css is imported once in main.jsx.

BEM-style class names (.product-card__title, .product-card--featured) provide scoping without CSS Modules or a compile step.

## Alternatives Considered

### SCSS

Nested selectors, mixins, @use partials, all preprocessed.

**Rejected because:** The advantages of SCSS over CSS have narrowed considerably. Custom properties handle variables. CSS nesting is now supported natively in modern browsers. SCSS adds a compile step and a dependency without providing capability that is missing.

### CSS Modules

Locally-scoped class names generated at build time.

**Rejected because:** Effective but adds noise to imports and JSX. For the size of this project, BEM discipline is sufficient.

### Tailwind CSS

Utility-first framework.

**Rejected because:** Would require the entire team to internalize Tailwind's utility system. Feature-specific CSS files are easier to review for a graded assignment.

### styled-components or Emotion

CSS-in-JS.

**Rejected because:** Runtime CSS-in-JS has real performance and bundle-size costs, and its ergonomics are contested. Plain CSS is faster and simpler.

## Consequences

### Positive

- Zero styling dependencies. No build-time preprocessing, no runtime CSS generation.
- CSS custom properties enable trivial theming (dark mode is a future one-line change).
- Feature-scoped stylesheets are easy to navigate — cart styles are in cart.css.

### Negative / Trade-offs

- No nesting (unless targeting only browsers with native CSS nesting support), no mixins, no @extend.
- Class-name discipline is manual. A typo in a BEM class produces a silent no-op rather than a build error.